

Here are the notes from the video, structured with subtitles, explanations, code examples, and outputs as requested. You can copy these directly into your copy.

Notes: Introduction to PyTorch

Subtitles:

1. What is PyTorch?
 2. The Origin Story: Torch & Lua
 3. The Birth of PyTorch: Torch + Python
 4. Key Timeline & Features
 5. Core Features of PyTorch
 6. PyTorch vs. TensorFlow: Which One to Choose?
 7. Core PyTorch Modules
 8. Ecosystem Libraries
 9. Industry Use Cases
 10. Roadmap for Learning PyTorch
-

1. What is PyTorch?

Explanation: PyTorch is an open-source, Python-based deep learning library. It allows you to build powerful neural network applications. It has become the de facto standard for deep learning, especially in the field of Generative AI and Large Language Models (LLMs).

2. The Origin Story: Torch & Lua

Explanation: Before PyTorch, there was a powerful scientific computing framework called **Torch**, released in 2002. Its main feature was performing **tensor-based operations** on GPUs. However, Torch was written in **Lua**, a programming language that was not very popular. This created a barrier for many developers who wanted to use it.

Key Problem with Torch:

1. **Lua-based:** Developers had to learn a new language (Lua) to use it.
 2. **Static Computation Graphs:** Once a model was built, its computational graph was fixed and could not be changed during runtime. This made experimentation and debugging difficult.
-

3. The Birth of PyTorch: Torch + Python

Explanation: Researchers at Meta AI (then Facebook AI) saw the problem. They merged the powerful capabilities of the Torch framework with the simplicity and popularity of the Python ecosystem. This marriage resulted in **PyTorch**.

The First Release (v0.1, 2017) introduced two revolutionary features:

- **Python Compatibility:** You could now write deep learning code in Python and it was compatible with other Python libraries (like NumPy).
- **Dynamic Computation Graphs:** Unlike Torch, PyTorch allowed the computational graph to be built and changed **on-the-fly** during runtime. This made it incredibly flexible and easier to debug.

Analogy for Dynamic vs. Static Computation Graphs:

- **Static (like old Torch/TensorFlow 1.x):** Imagine a government agency making a fixed map for a road. The road *must* be built exactly as per the map, even if there's a temple in the way. You can't change it.
 - **Dynamic (like PyTorch):** Imagine a team building a road that can make minor adjustments in real-time. If they encounter a temple, they can flexibly change the route. This is dynamic computation, allowing for more flexibility and experimentation.
-

4. Key Timeline & Features

Explanation: PyTorch's journey has been marked by key releases that expanded its capabilities from a research-focused tool to a production-ready framework.

Year	Version	Key Features
2017	0.1	Python compatibility, Dynamic Computation Graphs .
2018	1.0	TorchScript (for model serialization and optimization), merged with Caffe2 (improving production capabilities).
2018+	Subsequent	Distributed Training support, ONNX (Open Neural Network Exchange) support for interoperability, Quantization (for model compression), and Ecosystem Libraries (torchvision, torchtext, torchaudio).
2022+	2.x	Major focus on optimizations for speed and performance.

Year	Version	Key Features
		Models are faster, use fewer resources, and are optimized for custom hardware like TPUs.

5. Core Features of PyTorch

Explanation: PyTorch's popularity is built on these six core features.

1. **Tensor Computations:** Efficiently performs operations on tensors (multi-dimensional arrays), which are the fundamental data structure in deep learning.
2. **GPU Acceleration:** Easily move tensors and computations to a GPU for massive speedups.
3. **Dynamic Computation Graphs:** The defining feature. The graph is built at runtime, making it flexible, intuitive, and easy to debug.
4. **Autograd (Automatic Differentiation):** This is the engine that calculates gradients automatically, which is essential for backpropagation and training neural networks.
5. **Distributed Training:** Allows you to train models across multiple machines and GPUs, enabling you to work with very large datasets.
6. **Interoperability:** Can work with other Python and deep learning libraries, especially through the **ONNX** format.

Code Example: Tensors and Autograd

```
import torch

# 1. Tensor Computations & GPU Acceleration
# Create a tensor
x = torch.tensor([2.0, 3.0], requires_grad=True) # requires_grad
enables autograd
print(f"Tensor: {x}")

# Perform an operation
y = x ** 2
print(f"Result: {y}")

# 2. Autograd (Automatic Differentiation)
# Compute gradients (derivative of y with respect to x)
y.sum().backward()
print(f"Gradients: {x.grad}") # dy/dx = 2*x -> [4.0, 6.0]

# 3. GPU Acceleration (if a GPU is available)
if torch.cuda.is_available():
    x_gpu = x.cuda()
    print("Tensor moved to GPU")
```

Output:

```
Tensor: tensor([2., 3.], requires_grad=True)
Result: tensor([4., 9.], grad_fn=<PowBackward0>)
Gradients: tensor([4., 6.])
```

6. PyTorch vs. TensorFlow: Which One to Choose?

Explanation: This is a common dilemma. Here's a comparative analysis to help you decide.

Aspect	PyTorch	TensorFlow	Verdict
Ease of Use	More intuitive, feels like writing Python code.	Can have a steeper learning curve, especially with older APIs.	PyTorch Wins
Deployment & Production	Strong, with tools like TorchScript and PyTorch Mobile.	Very mature and robust with tools like TF Serving, TF Lite, and TF.js.	TensorFlow Wins
Performance	Excellent. Dynamic graphs were once slower but are now highly optimized.	Excellent. Eager execution (dynamic) is now default, making it comparable to PyTorch.	Tie
Research & Experimentation	The clear favorite in academia and research due to its flexibility and Pythonic nature.	Strong, but PyTorch has a larger share in cutting-edge research.	PyTorch Wins
Ecosystem	Hugging Face (transformers), PyTorch Lightning, etc. are incredibly powerful.	TF Hub, Keras, and a massive established community.	Tie
Industry Adoption	Rapidly growing, especially for Gen AI and LLMs.	Still dominant in many traditional industries and production environments.	Tie (but shifting)
Learning Curve	Easier.	Harder.	PyTorch Wins
Customization	Very high. Easy to create custom layers and operations.	Good, but can be more complex.	PyTorch Wins

The Final Verdict:

If your goal is to get into **Generative AI, LLMs, and cutting-edge research**, **PyTorch is the answer**. The boom in AI is largely driven by PyTorch.

If your goal is to work in a traditional industry environment focused on deploying conventional deep learning models at scale, TensorFlow is still a strong choice.

For a beginner targeting Gen AI, the recommendation is to focus on PyTorch.

7. Core PyTorch Modules

Explanation: When you install PyTorch, these are the main modules you'll interact with.

Module	Purpose
<code>torch</code>	The main module. Contains tensors, math operations, etc.
<code>torch.autograd</code>	Automatic differentiation engine for calculating gradients.
<code>torch.nn</code>	Building blocks for neural networks (layers, activation functions, loss functions).
<code>torch.optim</code>	Optimization algorithms (SGD, Adam, RMSprop).
<code>torch.utils.data</code>	Utilities for data loading (<code>Dataset</code> , <code>DataLoader</code>).
<code>torch.cuda</code>	For GPU acceleration.
<code>torch.quantization</code>	For compressing models by reducing numerical precision.
<code>torch.onnx</code>	For exporting models to the ONNX format for interoperability.

8. Ecosystem Libraries

Explanation: These are domain-specific libraries built on top of PyTorch.

- **`torchvision`**: For computer vision tasks. Contains datasets, models (like ResNet), and image transformations.
 - **`torchaudio`**: For audio and speech processing.
 - **`torchtext`**: For natural language processing. (Note: Hugging Face Transformers is now more popular for this domain).
 - **`PyTorch Lightning`**: A high-level API that simplifies the training loop and code organization.
 - **`Hugging Face Transformers`**: The go-to library for working with pre-trained Transformer models (LLMs, BERT, etc.). It is built on PyTorch.
-

9. Industry Use Cases

Explanation: Many leading tech companies use PyTorch in their products.

- **Microsoft:** Uses PyTorch extensively in its Azure machine learning services.
 - **Tesla:** Uses PyTorch for their Autopilot and computer vision systems.
 - **OpenAI:** Uses PyTorch to train their GPT models.
 - **Uber:** Uses PyTorch for demand forecasting, route optimization, and even created a programming language called **Pyro** (probabilistic programming) on top of it.
-

10. Roadmap for Learning PyTorch (This Playlist)

Explanation: The video outlines a clear learning path to master PyTorch.

1. **Fundamentals:**
 - Tensor basics and operations.
 - Understanding computation graphs.
 - The `autograd` package.
2. **Neural Networks:**
 - Building an Artificial Neural Network (ANN).
 - Building a Convolutional Neural Network (CNN).
 - Building a Recurrent Neural Network (RNN).
3. **Advanced Topics:**
 - Using PyTorch for model serving (TorchServe).
 - Model serialization with TorchScript.
 - Exporting models to ONNX format.
 - Model quantization for compression.

Here are the detailed notes from the video, structured topic-wise with explanations, code, and outputs where applicable. You can copy these directly into your notebook.

Topic 1: What are Tensors? (Theory)

- **Definition:** A tensor is a specialized **multidimensional array** designed for mathematical and computational efficiency. It is a generalization of arrays (1D) and matrices (2D) to any number of dimensions.
 - **Dimension (Rank):** The number of directions in which the tensor spans.
 - **0D (Scalar):** A single number. *Example:* The loss value from a forward pass.
 - **1D (Vector):** An array of numbers. *Example:* Word embeddings.
 - **2D (Matrix):** A grid of numbers. *Example:* A grayscale image.
 - **3D:** A cube of numbers. *Example:* An RGB image (height, width, color channel).
 - **4D:** A batch of RGB images (batch size, height, width, channels).
 - **5D:** A batch of video clips (batch size, frames, height, width, channels).
-

Topic 2: Why are Tensors Important in Deep Learning?

1. **Efficient Mathematical Operations:** Tensors are optimized for operations like addition, multiplication, and dot products, which are fundamental to neural networks.
 2. **Data Representation:** All real-world data (text, images, audio, video) can be easily represented as tensors.
 3. **Hardware Acceleration:** Tensors are designed to run on **GPUs (Graphics Processing Units)** and TPUs. GPUs have thousands of cores that can perform calculations in parallel, leading to massive speedups (e.g., 100x) compared to CPUs.
-

Topic 3: Practical - Setting up Environment & Checking GPU

Before starting, set up your environment (e.g., Google Colab) and check for GPU availability.

Code:

```
import torch

# Check PyTorch version
print("PyTorch Version:", torch.__version__)

# Check if GPU is available
print("Is CUDA (GPU) available?", torch.cuda.is_available())

# If GPU is available, get its name
if torch.cuda.is_available():
    print("GPU Name:", torch.cuda.get_device_name(0))
else:
    print("GPU not available. Using CPU.")
```

Output (example):

```
PyTorch Version: 2.5.1
Is CUDA (GPU) available? True
GPU Name: Tesla T4
```

Topic 4: Creating Tensors

Here are the most important methods to create tensors.

Code:

```
import torch

# 1. torch.empty(): Allocates memory without initializing (values are
# whatever was in memory).
a = torch.empty(2, 3) # Creates a 2x3 tensor
```

```

print("torch.empty:\n", a)

# 2. torch.zeros(): Creates a tensor filled with zeros.
b = torch.zeros(2, 3)
print("\ntorch.zeros:\n", b)

# 3. torch.ones(): Creates a tensor filled with ones.
c = torch.ones(2, 3)
print("\ntorch.ones:\n", c)

# 4. torch.rand(): Creates a tensor with random values between 0 and 1.
d = torch.rand(2, 3)
print("\ntorch.rand:\n", d)

# 5. For reproducibility with random numbers, set a seed.
torch.manual_seed(100)
e = torch.rand(2, 3)
print("\ntorch.rand with seed 100:\n", e)

# 6. torch.tensor(): Creates a tensor from a Python list or NumPy array.
f = torch.tensor([[4, 5, 6], [7, 8, 9]])
print("\ntorch.tensor from list:\n", f)

# 7. Other useful methods
g = torch.arange(0, 10, 2) # torch.arange(start, end, step)
print("\ntorch.arange:\n", g)

h = torch.linspace(0, 10, 5) # torch.linspace(start, end, steps)
print("\ntorch.linspace:\n", h)

i = torch.eye(3) # Identity matrix
print("\ntorch.eye:\n", i)

j = torch.full((2, 3), 5) # Tensor of shape 2x3 filled with value 5
print("\ntorch.full:\n", j)

```

Output:

```

torch.empty:
  tensor([[1.4013e-45, 0.0000e+00, 0.0000e+00],
          [0.0000e+00, 0.0000e+00, 0.0000e+00]])

torch.zeros:
  tensor([[0., 0., 0.],
          [0., 0., 0.]])

torch.ones:
  tensor([[1., 1., 1.],

```



```

    [1., 1., 1.]])

torch.rand:
  tensor([[0.6422, 0.2215, 0.3848],
          [0.3582, 0.1123, 0.3398]])

torch.rand with seed 100:
  tensor([[0.6965, 0.0333, 0.8178],
          [0.8791, 0.1456, 0.3459]])

torch.tensor from list:
  tensor([[4, 5, 6],
          [7, 8, 9]])

torch.arange:
  tensor([0, 2, 4, 6, 8])

torch.linspace:
  tensor([ 0.0000,  2.5000,  5.0000,  7.5000, 10.0000])

torch.eye:
  tensor([[1., 0., 0.],
          [0., 1., 0.],
          [0., 0., 1.]])

torch.full:
  tensor([[5, 5, 5],
          [5, 5, 5]])

```

Topic 5: Tensor Shape and Data Type

Code:

```

x = torch.tensor([[4, 5, 6], [7, 8, 9]])

# 1. Getting shape
print("Shape of x:", x.shape) # or x.size()
print("Number of dimensions (rank):", x.dim())
print("Total number of elements:", x.numel())

# 2. Getting data type
print("Data type of x:", x.dtype)

# 3. Specifying data type during creation
int_tensor = torch.tensor([1, 2, 3], dtype=torch.int32)
float_tensor = torch.tensor([1, 2, 3], dtype=torch.float32)
print("\nInteger Tensor:", int_tensor)
print("Float Tensor:", float_tensor)

```

```
# 4. Changing data type of an existing tensor
x_float = x.to(torch.float32)
print("\nOriginal x dtype:", x.dtype)
print("New x_float dtype:", x_float.dtype)

# 5. Using the `like` functions
y = torch.rand_like(x_float) # Creates a random tensor of same shape
as x_float
print("\nrand_like (float):\n", y)
```

Output:

```
Shape of x: torch.Size([2, 3])
Number of dimensions (rank): 2
Total number of elements: 6
Data type of x: torch.int64

Integer Tensor: tensor([1, 2, 3], dtype=torch.int32)
Float Tensor: tensor([1., 2., 3.])

Original x dtype: torch.int64
New x_float dtype: torch.float32

rand_like (float):
  tensor([[0.6437, 0.4945, 0.9355],
          [0.0399, 0.0482, 0.8516]])
```

Topic 6: Mathematical Operations

6.1 Scalar Operations

Operations between a tensor and a single number.

Code:

```
x = torch.tensor([1, 2, 3, 4], dtype=torch.float32)
print("Original x:", x)

print("x + 2:", x + 2)
print("x - 2:", x - 2)
print("x * 2:", x * 2)
print("x / 2:", x / 2)
print("x ** 2 (power):", x ** 2)
```

Output:

```
Original x: tensor([1., 2., 3., 4.])
x + 2: tensor([3., 4., 5., 6.])
x - 2: tensor([-1., 0., 1., 2.])
x * 2: tensor([2., 4., 6., 8.])
x / 2: tensor([0.5000, 1.0000, 1.5000, 2.0000])
x ** 2 (power): tensor([ 1.,  4.,  9., 16.])
```

6.2 Element-wise Operations

Operations between two tensors of the same shape.

Code:

```
a = torch.tensor([1, 2, 3])
b = torch.tensor([4, 5, 6])

print("a + b:", a + b)
print("a * b:", a * b)
print("a / b:", a / b)
```

Output:

```
a + b: tensor([5, 7, 9])
a * b: tensor([ 4, 10, 18])
a / b: tensor([0.2500, 0.4000, 0.5000])
```

6.3 Other Element-wise Functions

Code:

```
t = torch.tensor([-1.9, 2.3, -3.7, 4.4])

print("Absolute:", torch.abs(t))
print("Negative:", torch.negative(t))
print("Round:", torch.round(t))
print("Ceil (upper integer):", torch.ceil(t))
print("Floor (lower integer):", torch.floor(t))

# Clamp: limits values to a range
print("Clamp between 2 and 3:", torch.clamp(t, min=2, max=3))
```

Output:

```
Absolute: tensor([1.9000, 2.3000, 3.7000, 4.4000])
Negative: tensor([ 1.9000, -2.3000,  3.7000, -4.4000])
Round: tensor([-2.,  2., -4.,  4.])
Ceil (upper integer): tensor([-1.,  3., -3.,  5.])
Floor (lower integer): tensor([-2.,  2., -4.,  4.])
Clamp between 2 and 3: tensor([2., 2.3, 3., 3.])
```

6.4 Reduction Operations

Code:

```
e = torch.tensor([[1, 2, 3], [4, 5, 6]], dtype=torch.float32)
print("Tensor e:\n", e)

print("Sum of all elements:", torch.sum(e))
print("Mean of all elements:", torch.mean(e))

# Sum over dimension 0 (rows -> column-wise sum)
print("Sum over dimension 0:", torch.sum(e, dim=0)) # Result shape (3)

# Sum over dimension 1 (columns -> row-wise sum)
print("Sum over dimension 1:", torch.sum(e, dim=1)) # Result shape (2)

print("Maximum value:", torch.max(e))
print("Position of maximum value (argmax):", torch.argmax(e)) #
Flattens tensor
```

Output:

```
Tensor e:
  tensor([[1., 2., 3.],
          [4., 5., 6.]])
Sum of all elements: tensor(21.)
Mean of all elements: tensor(3.5000)
Sum over dimension 0: tensor([5., 7., 9.])
Sum over dimension 1: tensor([ 6., 15.])
Maximum value: tensor(6.)
Position of maximum value (argmax): tensor(5)
```

6.5 Matrix Operations

Code:

```
# Matrix Multiplication (mm)
f = torch.tensor([[1, 2, 3], [4, 5, 6]]) # Shape (2, 3)
g = torch.tensor([[7, 8], [9, 10], [11, 12]]) # Shape (3, 2)
print("Matrix Multiplication (2x3 * 3x2 = 2x2):\n", torch.mm(f, g))

# Dot Product (vectors)
v1 = torch.tensor([1, 2])
v2 = torch.tensor([3, 4])
print("\nDot Product (1*3 + 2*4):", torch.dot(v1, v2))

# Transpose
print("\nOriginal f:\n", f)
print("Transpose of f:\n", f.t())
```

Output:

```
Matrix Multiplication (2x3 * 3x2 = 2x2):
  tensor([[ 58,  64],
         [139, 154]])

Dot Product (1*3 + 2*4): tensor(11)

Original f:
  tensor([[1, 2, 3],
         [4, 5, 6]])
Transpose of f:
  tensor([[1, 4],
         [2, 5],
         [3, 6]])
```

Topic 7: In-Place Operations (The `_` Suffix)

In-place operations modify the original tensor directly, saving memory.

Code:

```
m = torch.tensor([1, 2, 3])
n = torch.tensor([4, 5, 6])

# Non-in-place: creates a new tensor
result = m + n
print("m after non-in-place addition:", m) # m is unchanged

# In-place: modifies m directly
m.add_(n) # Equivalent to m = m + n, but modifies m in-place
print("m after in-place addition (add_):", m)

# In-place ReLU activation
a = torch.tensor([-1, 2, -3, 4])
a.relu_() # Replaces negative values with 0
print("\na after in-place ReLU (relu_):", a)
```

Output:

```
m after non-in-place addition: tensor([1, 2, 3])
m after in-place addition (add_): tensor([5, 7, 9])

a after in-place ReLU (relu_): tensor([0, 2, 0, 4])
```

Topic 8: Copying Tensors (Clone vs. Assignment)

- **Assignment (=):** Creates a new reference to the same memory location. Changes to one affect the other.
- **clone():** Creates a new, independent copy in a new memory location.

Code:

```
a = torch.tensor([1, 2, 3])
print("Original a:", a)

# Shallow copy using assignment
b = a
print("b = a, b:", b)

# Modify a
a[0] = 100
print("\na after change:", a)
print("b after a changed:", b) # b changes because it points to the
same memory

# Deep copy using clone
c = a.clone()
a[1] = 200
print("\na after further change:", a)
print("c after a changed (clone):", c) # c remains unchanged

print("\nMemory IDs:")
print("ID of a:", id(a))
print("ID of b (assigned):", id(b)) # Same as a
print("ID of c (cloned):", id(c)) # Different from a
```

Output:

```
Original a: tensor([1, 2, 3])
b = a, b: tensor([1, 2, 3])

a after change: tensor([100,  2,  3])
b after a changed: tensor([100,  2,  3])

a after further change: tensor([100, 200,  3])
c after a changed (clone): tensor([100,  2,  3])

Memory IDs:
ID of a: 140213469938640
ID of b (assigned): 140213469938640
ID of c (cloned): 140213469939376
```

Topic 9: Moving Tensors to GPU

Code:

```
# 1. Check if GPU is available
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print("Using device:", device)

# 2. Create a tensor directly on the GPU
gpu_tensor = torch.rand(2, 3, device=device)
print("\nTensor created on GPU:\n", gpu_tensor)
print("Device of gpu_tensor:", gpu_tensor.device)

# 3. Move an existing CPU tensor to GPU
cpu_tensor = torch.rand(2, 3)
print("\nCPU tensor device:", cpu_tensor.device)
gpu_tensor_2 = cpu_tensor.to(device)
print("GPU tensor device after .to():", gpu_tensor_2.device)

# 4. Speed comparison (example code)
if torch.cuda.is_available():
    import time
    size = 10000
    cpu_a = torch.rand(size, size)
    cpu_b = torch.rand(size, size)

    # CPU Computation
    start = time.time()
    cpu_result = torch.mm(cpu_a, cpu_b)
    cpu_time = time.time() - start

    # GPU Computation
    gpu_a = cpu_a.to(device)
    gpu_b = cpu_b.to(device)
    start = time.time()
    gpu_result = torch.mm(gpu_a, gpu_b) # Wait for GPU to finish
    torch.cuda.synchronize() # Ensure GPU computation is done
    gpu_time = time.time() - start

    print(f"\nCPU time for {size}x{size} matmul: {cpu_time:.2f} seconds")
    print(f"GPU time for {size}x{size} matmul: {gpu_time:.2f} seconds")
    print(f"Speedup: {cpu_time/gpu_time:.2f}x")
```

Output (example):

Using device: cuda

Tensor created on GPU:

```
tensor([[0.7129, 0.7220, 0.8194],
        [0.6988, 0.6017, 0.6268]], device='cuda:0')
Device of gpu_tensor: cuda:0

CPU tensor device: cpu
GPU tensor device after .to(): cuda:0

CPU time for 10000x10000 matmul: 17.35 seconds
GPU time for 10000x10000 matmul: 0.58 seconds
Speedup: 29.91x
```

Topic 10: Reshaping Tensors

Code:

```
# Create a tensor of shape (4, 4)
r = torch.arange(16).reshape(4, 4)
print("\nOriginal tensor (4x4):", r)

# 1. reshape()
reshaped = r.reshape(2, 8)
print("\nReshape to (2, 8):", reshaped)

# 2. flatten()
flat = r.flatten()
print("\nFlattened (1D):", flat)

# 3. permute() - reorders dimensions
# Create a 3D tensor (2, 3, 4)
p = torch.arange(24).reshape(2, 3, 4)
print("\nOriginal 3D shape (2,3,4):", p.shape)

# Permute dimensions: move dimension 2 (index 2) to front, then 0,
# then 1
permuted = p.permute(2, 0, 1)
print("\nPermuted shape (4,2,3):", permuted.shape)

# 4. unsqueeze() - adds a dimension of size 1
img = torch.rand(226, 226, 3) # Example image shape
print("\nImage shape:", img.shape)
batch_img = img.unsqueeze(0) # Add batch dimension at position 0
print("\nAfter unsqueeze (batch added):", batch_img.shape) # (1, 226, 226, 3)

# 5. squeeze() - removes dimensions of size 1
batch_removed = batch_img.squeeze(0)
print("\nAfter squeeze (batch removed):", batch_removed.shape)
```


Output:

```
Original tensor (4x4):
  tensor([[ 0,  1,  2,  3],
          [ 4,  5,  6,  7],
          [ 8,  9, 10, 11],
          [12, 13, 14, 15]])

Reshape to (2, 8):
  tensor([[ 0,  1,  2,  3,  4,  5,  6,  7],
          [ 8,  9, 10, 11, 12, 13, 14, 15]])

Flattened (1D): tensor([ 0,  1,  2,  3,  4,  5,  6,  7,  8,  9, 10,
 11, 12, 13, 14, 15])

Original 3D shape (2,3,4): torch.Size([2, 3, 4])
Permuted shape (4,2,3): torch.Size([4, 2, 3])

Image shape: torch.Size([226, 226, 3])
After unsqueeze (batch added): torch.Size([1, 226, 226, 3])
After squeeze (batch removed): torch.Size([226, 226, 3])
```

Topic 11: Interoperability with NumPy

Code:

```
import numpy as np

# 1. PyTorch Tensor -> NumPy Array
tensor = torch.tensor([1, 2, 3])
print("PyTorch Tensor:", tensor)
numpy_array = tensor.numpy()
print("Converted NumPy Array:", numpy_array)
print("Type:", type(numpy_array))

# 2. NumPy Array -> PyTorch Tensor
np_array = np.array([4, 5, 6])
print("\nNumPy Array:", np_array)
tensor_from_np = torch.from_numpy(np_array)
print("Converted PyTorch Tensor:", tensor_from_np)
print("Type:", type(tensor_from_np))
```

Output:

```
PyTorch Tensor: tensor([1, 2, 3])
Converted NumPy Array: [1 2 3]
Type: <class 'numpy.ndarray'>
```

```
NumPy Array: [4 5 6]
Converted PyTorch Tensor: tensor([4, 5, 6])
Type: <class 'torch.Tensor'>
```

PyTorch Autograd Notes

1. Need for Autograd

Explanation

- **Autograd** = Automatic Differentiation in PyTorch. It calculates derivatives automatically.
- Derivatives are crucial for training neural networks (backpropagation).
- **Problem:** As neural networks become deeper (nested functions), manually computing derivatives using chain rule becomes extremely difficult.

Neural Network as Nested Function

A simple neural network (single neuron with sigmoid) involves:

1. `z = w * x + b`
2. `y_pred = sigmoid(z)`
3. `Loss = BinaryCrossEntropy(y_pred, y_true)`

Here, Loss is a nested function of w and b. To update w and b, we need:

- $\partial L / \partial w$ and $\partial L / \partial b$ using chain rule:
 - $\partial L / \partial w = \partial L / \partial y_{\text{pred}} * \partial y_{\text{pred}} / \partial z * \partial z / \partial w$
 - $\partial L / \partial b = \partial L / \partial y_{\text{pred}} * \partial y_{\text{pred}} / \partial z * \partial z / \partial b$

Manually coding these for large networks is impractical. **Autograd automates this.**

2. Autograd Basics

Explanation

- Set `requires_grad=True` on a tensor to track operations on it.
- After forward pass, call `.backward()` on the output tensor.
- Gradients are stored in `.grad` attribute of the input tensor.
- PyTorch builds a **computation graph** (Directed Acyclic Graph) during forward pass.
- Backward pass traverses the graph from root (output) to leaves (inputs) applying chain rule.